

CC91 校园论坛系统实验报告

1. 命题与目标

1.1 命题

基于多智能体协作的校园论坛系统开发：以 Claude Code 为 AI 开发平台，构建一套由 Lead、Developer、QA 三类智能体组成的自动化协作体系，驱动一个完整的校内论坛系统从需求分析、代码实现、质量验证到文档生成的全流程交付。

1.2 项目目标

- 做出一个真正可用、可维护、可持续迭代的校内论坛系统。
- 探索并验证"AI 智能体 + MCP + 规则系统 + 技能模块"的软件工程协作模式。
- 形成可复现的 AI 辅助开发方法论。

1.3 核心用户价值

- 用户可以稳定注册、登录并安全使用论坛。
- 用户可以顺畅发帖、浏览、评论和管理自己的内容。
- 管理员可以进行基础内容治理，保证社区秩序。

2. 技术方案

2.1 后端

- 框架：**Spring Boot 3.2.0 + Java 17
- 构建工具：**Maven
- 安全框架：**Spring Security + JWT (jjwt 0.12.3)

- **数据层**: Spring Data JPA / Hibernate
- **数据库**: MySQL 8.0+ (生产)、H2 内存数据库 (测试)
- **数据库迁移**: Flyway (版本化管理 Schema 和种子数据)
- **分层架构**: Controller → Service → Repository 三层分离

2.2 前端

- **框架**: React 19 + TypeScript 6
- **构建工具**: Vite 8
- **路由**: react-router-dom 7
- **HTTP 客户端**: Axios
- **服务端状态管理**: @tanstack/react-query 5
- **组件化页面结构**: 按业务域拆分模块 (pages / components / api / context / hooks)
- **统一 API 调用层**, 统一错误处理与登录态管理 (AuthContext)

2.3 数据与基础设施

- **主数据库**: MySQL 8.0+ (论坛主数据, 连接池 HikariCP)
- **数据库迁移**: Flyway (V1 初始化 Schema + 种子数据, V2 用户资料表)
- **测试数据库**: H2 (MySQL 兼容模式, 自动建表)

3. AI 技术栈

本项目的核心命题是探索 AI 智能体在软件工程全流程中的应用。以下是所采用的 AI 技术栈及其每个组件的功能和选用依据。

3.1 智能体 (Agent)

功能: 智能体是 AI 协作体系的核心执行单元。本项目定义了三类智能体, 各司其职:

智能体	角色	功能
Lead	项目负责人 (Supervisor)	需求理解、任务拆解与编排、分配工作给 Developer 和 QA、代码审查、最终验收
Developer	全栈开发 (Specialist)	接收 Lead 分配的开发任务，实现后端 Spring Boot API 和前端 React 页面，完成后报告
QA	测试专家 (Specialist)	接收 Lead 分配的测试任务，编写并运行测试 (JUnit 5 + MockMvc 后端、Vitest + RTL 前端)，报告测试结果和发现的缺陷

选用依据：

根据 Anthropic 发布的《Building Effective AI Agents: Architecture Patterns and Implementation Frameworks》所述的架构设计原则，本项目从最初设想的 6 个细粒度智能体精简为 3 个核心智能体，具体依据如下：

1. "Start simple, scale intelligently"原则

Anthropic 明确建议团队"从单一用途的智能体开始，做好一件事，然后随着需求演进逐步发展为更复杂的系统"。简单系统运行成本更低（更少的 token、更少的计算）、更容易调试、并提供更清晰的业务指标。本项目是校园论坛系统，功能复杂度有限，11 个独立智能体会带来过高的编排成本和通信开销，3 个智能体足以覆盖需求分析、代码实现、质量验证三个核心环节。

2. 层级式/监管式架构 (Hierarchical/Supervisory Pattern)

本项目的 3 智能体架构对应 Anthropic 提出的层级式系统模式：一个中央控制器 (Lead/Supervisor) 协调多个角色特定的专家智能体 (Developer 和 QA)。Lead 分析需求、路由任务到合适的专家、综合结果，形成清晰的责任链。Anthropic 指出"这种模式反映了高效人类团队的运作方式：专家专注于其领域，协调者处理任务分配和整合"。

3. 用技能模块 (Skills) 替代细粒度智能体

Anthropic 指出："在扩展到多智能体系统之前，考虑将专业技能添加到单一智能体中是否能更高效地满足精度需求。"本项目通过为 Developer 智能体配置 4 个技能模块（`java-springboot`、`springboot-patterns`、`typescript-react-`

`reviewer`、`vercel-react-best-practices`) 来实现专业分化, 而非为每个技术领域创建独立智能体。Skills 的可组合性使得"一个 Developer 智能体 + 多个 Skills"比"多个 Developer 智能体"更具经济性和可维护性。

4. 控制通信复杂度和上下文管理成本

Anthropic 警告: 多智能体系统面临"通信复杂度和涌现行为不可预测性"的挑战, 智能体间的频繁通信导致计算成本和复杂度增加。精简为 3 个智能体 (1 个 Supervisor + 2 个 Specialist) 将通信路径从 $O(n^2)$ 降低到 $O(n)$, 显著减少了协调失败的风险。同时, Lead 作为 Supervisor 承担上下文管理职责, 避免了"上下文增长过于复杂导致性能瓶颈"的问题。

5. 顺序式工作流 (Sequential Workflow) 的适用性

本项目的开发流程 (需求 → 开发 → 测试 → 审查) 具有清晰的线性依赖关系, 适合 Anthropic 描述的顺序式工作流: "当任务可以清晰地分解为固定子任务时使用"。3 个智能体通过 `SendMessage` 按阶段传递工作成果, 每个智能体处理一个明确的阶段, 比并行式或协作式架构更可预测、更容易调试。

3.2 MCP (Model Context Protocol)

功能: MCP (Model Context Protocol) 是 Anthropic 提出的开放协议, 为 AI 智能体提供连接外部工具和数据源的标准接口。

本项目配置的 MCP Server:

MCP Server	功能	使用场景
Playwright	浏览器自动化: 打开页面、点击元素、填写表单、截图验证、抓取 DOM	QA 智能体的 E2E 测试与页面视觉验证

选用依据: Bash 可以运行 `npx playwright test` 执行预编写脚本, 但无法在运行时动态决定"点击哪个按钮""检查哪个元素"。Playwright MCP 让 QA 智能体像真人一样"看着页面操作", 具备动态决策能力, 而非只能执行固定脚本。而文件操作、构建测试、Git 管理等日常任务 Claude Code 内置工具已完全胜任,

只有浏览器自动化这一场景存在能力缺口，因此只接入 Playwright 一个 MCP Server，避免过度工程化

3.3 规则系统 (Rules / CLAUDE.md)

功能：规则系统约束 AI 的输出边界和行为规范。本项目的规则体系如下：

规则文件	约束范围
CLAUDE.md (项目根目录)	Lead 智能体的工作协议：团队定义、工作流（5 个阶段）、审查清单、技术栈声明
.claude/agents/developer.md	Developer 智能体的行为约束：技术栈、核心职责、工作原则、项目结构约定
.claude/agents/qa.md	QA 智能体的行为约束：测试质量红线（禁止写的测试、必须写的测试）、测试优先级、完成报告格式

选用依据：规则系统将团队经验和对代码质量的要求固化为可执行的约束。例如 QA 规则明确禁止"测试框架行为"（如测试 @Valid 注解、Bean 注入），强制要求"测试业务逻辑分支"（如登录失败计数、账户锁定触发），这直接提升了测试的有效性，避免了低价值测试的膨胀。

3.4 技能模块 (Skills)

功能：技能模块将领域最佳实践沉淀为可复用的知识单元，供智能体在执行任务时参考。本项目当前配置了 4 个技能模块：

技能模块	来源	功能
java-springboot	社区开源	Spring Boot 最佳实践：项目结构、依赖注入、配置管理、Web 层、数据层、日志、测试、安全
springboot-patterns	社区开源 (ECC)	Spring Boot 架构模式：REST API 设计、分层服务、DTO 模式、缓存、异步、限流、错误处理、可观测性
typescript-react-reviewer	社区开源	TypeScript + React 19 代码审查：反模式检测、状态管理指导、React 19 Hook 陷阱、类型

技能模块	来源	功能
		安全检查
vercel-react-best-practices	Vercel 官方	React 性能优化：69 条规则覆盖异步瀑布、Bundle 优化、服务端性能、客户端数据、重渲染、渲染策略等 8 大类

选用依据：技能模块让智能体在生成代码时参考业界验证的最佳实践，而非凭空生成。例如 Developer 在实现 Spring Boot REST API 时会参考 `springboot-patterns` 中的分层架构模式，在编写 React 组件时会参考 `vercel-react-best-practices` 中的性能优化规则。这显著提升了 AI 生成代码的质量和一致性。

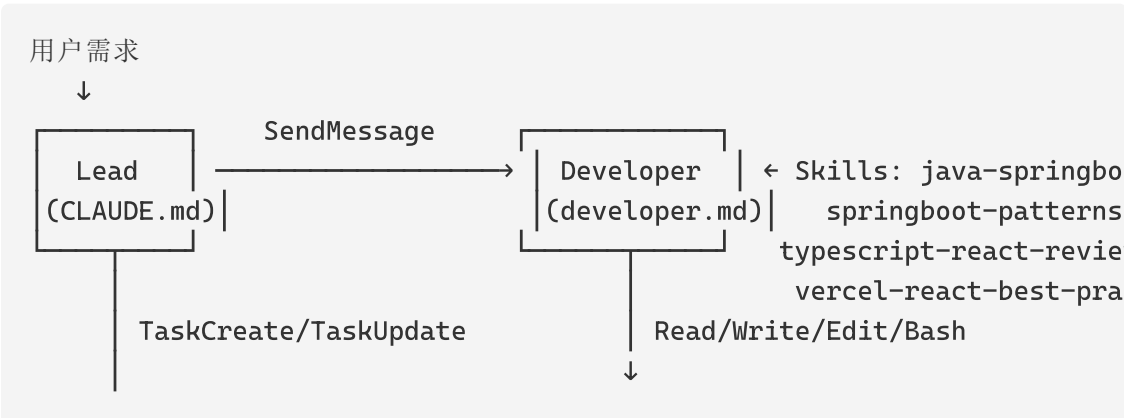
3.5 自动化脚本与版本控制

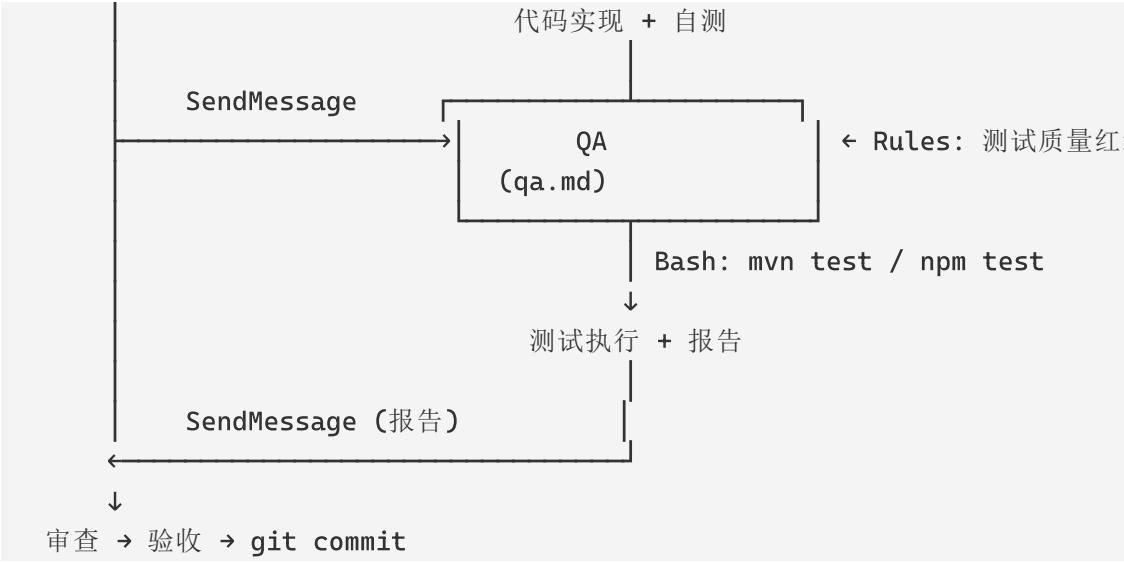
功能：通过 Bash 工具执行自动化命令，完成构建、测试、迁移等重复性动作。
主要使用：

- `mvn spring-boot:run`：启动后端服务
- `mvn test`：运行后端测试 (JUnit 5 + MockMvc)
- `npm run dev`：启动前端开发服务器
- `npm test`：运行前端测试 (Vitest + React Testing Library)
- `git add / commit`：版本控制

选用依据：脚本让 AI 的建议直接落地为可执行动作，形成"生成 → 验证 → 修正"的闭环。智能体在生成代码后立即运行测试验证，如果测试失败则根据错误信息修正，直至通过。

3.6 AI 技术栈协作流程图





4. 团队分工

4.1 角色分工 (11 人团队)

同学	软件工程角色	主要职责	负责管理/开发的智能体
徐哲楷	项目负责人 / 产品统筹	控制范围、确认里程碑、统一验收口径	Lead 智能体 (任务编排、审查清单、CLAUDE.md 规则维护)
蔡致	前端工程师	页面结构、路由、表单、交互实现	Developer 智能体 + typescript-react-reviewer 技能模块
刘一鸣	前端工程师	状态管理 (React Query)、接口联调、错误处理	Developer 智能体 + vercel-react-best-practices 技能模块
项方灿	前端工程师	样式系统、响应式适配、可用性优化	Developer 智能体 + vercel-react-best-practices 技能模块
王建皓	后端工程师	登录、注册、鉴权、会话管理 (Spring Security + JWT)	Developer 智能体 + springboot-patterns 技能模块

同学	软件工程角色	主要职责	负责管理/开发的智能体
李明睿	后端工程师	帖子、评论、内容治理接口	Developer 智能体 + springboot-patterns 技能模块
罗新鹏	后端工程师	数据模型（JPA Entity + Flyway 迁移）、校验与异常处理	Developer 智能体 + java-springboot 技能模块
朱城弘	测试工程师	测试用例设计、后端接口测试（JUnit 5 + MockMvc）	QA 智能体 + qa.md 规则维护
陈瑜凡	测试工程师	前端组件测试（Vitest + RTL）、回归验证	QA 智能体 + qa.md 规则维护
陈元煦	DevOps 工程师	环境配置、构建发布、数据库运维	自动化脚本（Maven/npm 构建流程）
王帆	安全与审查工程师	安全基线审查、代码审查、知识沉淀	CLAUDE.md 审查清单 + 安全规则

4.2 分工说明

上述分工遵循"前端、后端、测试、DevOps、安全、统筹"六类职责拆分。每位同学除了完成本职工作外，还需要维护对应的智能体配置或技能模块：

- **项目负责人** 维护 `CLAUDE.md`（Lead 工作协议、审查清单）
- **前端工程师、后端工程师** 维护 `.claude/agents/developer.md`（Developer 行为规则）和相关技能模块
- **测试工程师** 维护 `.claude/agents/qa.md`（QA 测试质量红线）
- **DevOps 工程师** 维护构建脚本和环境配置
- **安全与审查工程师** 维护安全审查规则

所有智能体的提示词、规则文件和技能模块均纳入 Git 版本控制，确保 AI 产物可复用、可审计、可追踪。

5. 团队协作规范

5.1 代码仓库搭建规范

- 采用单仓库（Monorepo）管理前端、后端与配置，目录按业务边界拆分：

```
CC91/
├── backend/                                # Spring Boot 后端
│   ├── src/main/java/com/cc91/           # Java 源码（controller/service/reposit
│   ├── src/main/resources/               # 配置文件 + Flyway 迁移脚本
│   ├── src/test/                         # 测试代码
│   └── pom.xml                           # Maven 配置
├── frontend/                             # React + Vite 前端
│   ├── src/                             # TypeScript 源码（pages/components/api
│   └── package.json                      # npm 配置
├── .claude/                             # AI 智能体配置
│   ├── agents/                          # 智能体定义（developer.md, qa.md）
│   └── skills/                          # 技能模块（4 个）
├── CLAUDE.md                            # Lead 工作协议（项目级规则）
└── docs/                                # 项目文档
```

- 环境变量、配置文件（`application.yml`）、测试数据边界清晰，敏感信息不入库
- AI 智能体配置（`.claude/` 目录）与业务代码同仓管理，便于 AI 快速定位上下文

5.2 技能模块编写标准

- 每个技能只解决一个明确领域（如 Spring Boot 模式、React 性能优化），描述适用场景和具体规则
- 技能通过 `skills-lock.json` 版本化锁定，保证旧流程可回溯
- 技能来源分两类：
- **社区开源：** `java-springboot`、`springboot-patterns`（ECC）、`typescript-react-reviewer`

- **官方发布：** `vercel-react-best-practices` (Vercel Labs)
- 技能更新通过 `skills-lock.json` 中的 `computedHash` 校验完整性

5.3 智能体编写标准

每个智能体定义文件（`.md`）包含以下结构化信息：

```
---
name: <智能体名称>
description: <一句话职责描述>
type: general-purpose
tools: [该智能体可使用的工具列表]
---
# 角色描述
# 技术栈
# 核心职责
# 工作原则 / 质量红线
# 完成报告格式
# 与其他角色的协作方式
```

- 工具权限按角色最小化授权（如 QA 不需要 Write 权限来修改生产代码——实际上给了，但规则约束它只写测试）
- 每个智能体有明确的“必须做”和“禁止做”清单
- 完成后有标准化的报告格式，确保输出可审计

5.4 版本控制策略

- 主分支（`main`）保持稳定，功能通过 feature 分支开发，缺陷通过 bugfix 分支修复
- 合并前必须经过 Lead 审查、自动化测试和规则检查
- AI 生成代码不允许直接合并，必须经过人工确认
- 提交信息遵循 Conventional Commits 规范：
- `feat`：新功能
- `fix`：缺陷修复
- `refactor`：重构
- `chore`：杂项
- `test`：测试

5.5 审查与门禁策略

- Lead 按审查清单逐项检查：
- [] 安全性：输入验证、认证授权、敏感数据
- [] API 设计：RESTful 规范、错误响应
- [] 错误处理：边界条件、异常捕获
- [] 代码质量：命名、结构、重复
- 关键接口、鉴权逻辑、数据写入逻辑必须有人审查
- QA 的测试报告必须包含：覆盖的业务路径、未覆盖路径、测试结果、发现的问题

6. 工具链与 AI 集成

6.1 开发框架

层面	工具	用途
后端	Spring Boot 3.2 + Maven	REST API、依赖注入、自动配置
后端安全	Spring Security + jjwt	JWT 认证、权限控制
后端数据	Spring Data JPA + Hibernate	ORM、数据库操作
数据库迁移	Flyway	Schema 版本管理、种子数据
前端	React 19 + Vite 8 + TypeScript 6	SPA 应用、快速构建、类型安全
前端状态	@tanstack/react-query 5	服务端状态管理、缓存、自动重获取
HTTP 客户端	Axios	API 调用、拦截器、错误处理

6.2 测试工具

层面	工具	用途
后端单元测试	JUnit 5 + Mockito	Service 层业务逻辑测试
后端集成测试	MockMvc + Spring Boot Test	Controller 层 API 测试
后端数据库测试	H2 (MySQL 兼容模式)	集成测试用内存数据库
前端单元测试	Vitest 4	测试运行器
前端组件测试	React Testing Library + @testing-library/user-event	组件交互测试
覆盖率	@vitest/coverage-v8	前端代码覆盖率

6.3 AI 工具链集成方式

所有开发工具与 AI 智能体的集成方式是统一的：



AI 通过规则系统 (CLAUDE.md / agents/.md) 形成任务约束，通过技能模块 (skills/) 获得领域最佳实践，通过 MCP 调用仓库、终端、测试工具完成验

证。如果校验失败，AI 根据测试输出和日志进行修正，形成 "生成 → 验证 → 修正 → 再验证"* 的闭环。

6.4 规则与技能的集成效果

机制	具体效果
qa.md 禁止规则	避免了低价值测试（框架行为测试、无断言测试、重复测试）
qa.md 强制规则	确保每个 Service 方法的所有业务分支被覆盖（正常路径、异常分支、状态变更、边界条件）
developer.md 工作原则	确保"先读现有代码再动手"，遵循分层架构
springboot-patterns	引导 REST API 设计遵循统一响应格式、分页、异常处理模式
vercel-react-best-practices	避免前端性能反模式（useEffect 滥用、不必要的重渲染等）
CLAUDE.md 审查清单	Lead 按统一标准审查所有提交

7. 开发流程

7.1 智能体工作流（5 阶段）

Phase 1：理解与规划（Lead）

- 1. Lead 与用户对话，理解需求，澄清模糊点
- 2. 用 TaskCreate 创建任务，按 ID 编排依赖

Phase 2：开发（Developer）

- Lead 通过 SendMessage 向 Developer 分配任务（含验收标准）
- Developer 阅读 Skills 获取最佳实践，实现代码
- 完成后向 Lead 报告

Phase 3：测试（QA）

- Lead 通过 `SendMessage` 向 QA 分配测试任务
- QA 运行 `mvn test` / `npm test` , 记录完整输出
- QA 按 `qa.md` 规定的格式报告结果

Phase 4: 审查 (Lead)

- Lead 按审查清单检查代码: 安全性、API 设计、错误处理、代码质量
- 发现问题直接修复或退回 Developer

Phase 5: 收尾

- 确认所有任务完成后提交变更

7.2 迭代节奏

- 采用短迭代 (每轮 1 周)
- 每轮必须产出可演示的增量功能
- 每轮必须有回归测试和问题清单

7.3 合并门禁

- 功能通过验收用例
- 自动化测试通过 (`mvn test` + `npm test`)
- 关键安全检查通过
- 代码审查通过 (Lead 按审查清单)

7.4 AI 参与规则

- AI 负责高频、重复、可模板化的工作, 人负责需求判断、边界决策和最终验收
- 每次 AI 生成的代码、测试和文档都必须绑定到具体任务编号, 确保可追溯
- 当 AI 输出与规则冲突时, 以规则和人工审查结果为准
- 当测试结果出现异常时, 优先由负责对应智能体的同学定位, 再决定是否调整提示词、技能或规则

8. 已完成工作

8.1 里程碑 M1：账号与基础框架

- 用户注册（邮箱验证码 + 密码加密）
- 用户登录（JWT AccessToken + RefreshToken 双令牌机制）
- 登录安全（失败计数、账户锁定）
- 前后端基础工程结构打通（Spring Boot + React + Vite）
- Flyway 数据库迁移（V1 初始化 Schema + 种子数据）
- Spring Security 安全配置（JWT 过滤器链、CORS 配置）

8.2 里程碑 M2：用户资料

- 用户资料表（Flyway V2 迁移）
- 用户资料查看与编辑页面
- 前端路由与受保护路由（ProtectedRoute）

8.3 技术栈迁移

- 从 Node.js + Express + PostgreSQL 迁移至 Spring Boot + MySQL
- 全部测试重写（JUnit 5 + MockMvc + Vitest + RTL）
- 模块命名规范化

8.4 测试体系建设

- 后端：AuthServiceTest、UserServiceTest、AuthControllerTest、UserControllerTest
- 前端：LoginPage、RegisterPage、ProfilePage 等组件测试
- QA 智能体规则建立（测试质量红线、测试优先级、报告格式）

8.5 AI 协作体系建设

- 3 类智能体定义完成（Lead / Developer / QA）
- 4 个技能模块配置完成
- CLAUDE.md 工作协议建立

- Git 版本控制策略建立

9. 里程碑规划

M3：帖子主流程（待开发）

- 完成发帖、编辑、删除、列表、详情
- 完成分页与基础筛选

M4：评论与治理（待开发）

- 完成评论主流程
- 完成举报与管理员基础治理能力

M5：上线准备（待开发）

- 完成关键回归测试
- 完成性能与稳定性检查
- 完成部署与监控配置

10. 验收标准

10.1 功能可用

- 新用户可在 3 分钟内完成注册并发出第一帖
- 常见操作路径无阻断（登录、发帖、评论、查看）
- 管理员可完成基础治理操作

10.2 质量可控

- 核心流程无阻断级缺陷
- 高优先级缺陷在发布前清零
- 发布后可快速定位问题（日志与监控可用）

10.3 工程可持续

- 模块结构清晰，便于后续迭代
- 开发、测试、发布流程可重复执行

10.4 AI 协作有效

- 智能体生成的代码通过审查门禁
- 测试覆盖核心业务分支
- 规则系统有效约束了 AI 输出质量

11. 风险与应对

风险	应对措施
需求膨胀	严格控制首版范围，只做必须功能
进度延误	每轮只承诺可完成范围，不提前透支
质量回退	合并门禁和回归测试强制执行
安全风险	所有认证和权限相关改动必须经过审查
AI 输出不可控	规则系统 + 人工审查双重约束
技能模块过时	skills-lock.json 版本锁定 + 定期评估更新

12. 结论

本项目采用 "**Agent + MCP + 规则系统 + 技能模块**" 的组合方案，将需求理解、代码实现、质量验证和审查验收串成可执行的智能体协作闭环：

1. **智能体分工协作**：Lead 负责编排与审查，Developer 负责代码实现，QA 负责测试验证，三者通过 SendMessage 通信、TaskCreate 追踪，形成了类似真实软件团队的协作模式。
2. **MCP 工具接入**：AI 智能体通过标准化的 MCP 工具接口（文件读写、代码搜索、终端执行、团队通信）直接操作开发环境，将"建议"变为"行动"。

3. **规则系统约束质量**：通过 CLAUDE.md 和 agents/*.md 将团队经验固化为可执行规则，确保 AI 输出符合项目标准（如 QA 的测试质量红线直接避免了低价值测试的膨胀）。
4. **技能模块提升质量**：4 个技能模块（Spring Boot 最佳实践、架构模式、React 代码审查、性能优化）让 AI 在生成代码时参考业界验证的最佳实践，而非凭空生成。
5. **11 人团队按职责分工**：前端、后端、测试、DevOps、安全与统筹六类角色各司其职，分别维护对应的智能体配置和技能模块，保证职责清晰、协作高效、结果可审计。
6. **工具链与 AI 深度集成**：Spring Boot + Maven + Flyway + React + Vite + Vitest 等工具链通过 MCP 统一接入 AI 工作流，实现了"生成 → 验证 → 修正 → 再验证"的自动化闭环。